גיליון תשובות

מספר נבחן:_____

שאלה 1: זיכרון (35 נקודות)

א.

```
template <typename T>
class copy_on_write {

public:
    T* ptr;

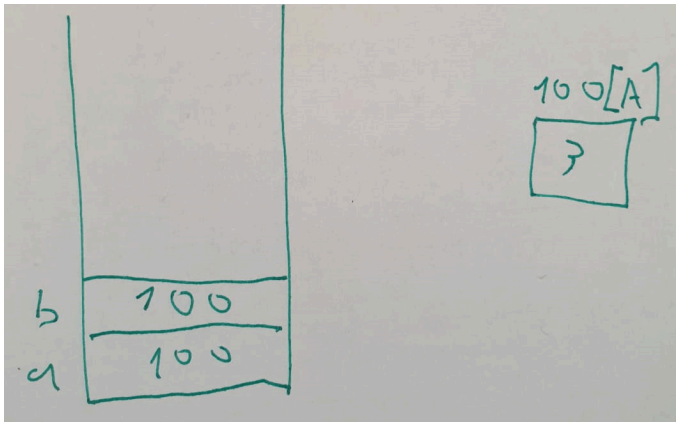
    copy_on_write(T* p) : ptr(p) {}

    void on_write() {
        ptr = ptr->clone();
    }
};
```

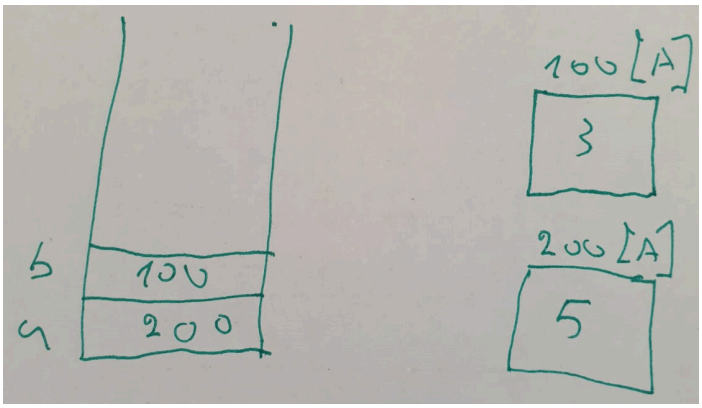
ב.

```
 {
    copy_on_write<A> a(new A(3));
    copy_on_write<A> b(a->ptr);
    //@@Draw the memory
    std::cout << b.ptr->get();
    a.on_copy();
    a.ptr->set(5);
    //@@Draw the memory
}
```

תמונת הזיכרון לפני ההשמה:



תמונת הזיכרון לאחר ההשמה:



ג. מאחר שיש רק מצביע אחד למקום הזיכרון בעקבות סיום הבלוק, אין צורך לשכפלו.

יש לממש מנגנון של referrence count, כפי שנעשה בתבנית ה-smart_ptr שנלמדת בכיתה, ולבצע שכפול ב-on-write רק האם המונה גדול ב-1.

```
template <typename T>
class copy_on_write {

public:
    T* ptr;
    int* rc;

    copy_on_write(T* p) : ptr(p), rc(new int(1)) {}
```

```
    copy_on_write(const copy_on_write& other) : ptr(other.ptr), rc(other.rc) {
        (*rc)++;
     }

    copy_on_write& operator=(const copy_on_write& other) {
        if (this != &other) {
            clean();
            ptr = other.ptr;
            rc = other.rc;
            (*rc)++;
        }
        return *this;
    }

    ~copy_on_write() { clean(); }

    void on_copy() {
        if ((*rc) > 1)
            ptr = ptr->clone();
    }

private:
    void clean() {
        (*rc)--;
        if ((*rc) == 0) {
            delete ptr;
            delete rc;
        }
    }
};
```

שאלה 2: מקביליות (30 נקודות)

א. מוסיפים סנכרון על this בכל מתודות ה-public.

ב. נוסיף מימוש נוסף של clone עם אייטם להוספה (התוספות על clone הרגילה מסומנות בקו תחתון):

```
public Set<T> clone(T item) {
        if (head == null)
            return new SetImpl(item);
        else {
            boolean bFound=false;
            Link<T> this_curr_link = head;
            Link<T> copy_curr_link = new Link<T>(this_curr_link.data,null);
            Set<T> ret = new SetImpl(copy_curr_link);
            while (this_curr_link.next != null) {
                if (item.equals(this_curr_link.data))
                    bFound = true;
                copy_curr_link.next = new Link<T>(this_curr_link.next.data,null);
                this_curr_link = this_curr_link.next;
                copy_curr_link = copy_curr_link.next;
            }
            if (!bFound)
                copy_curr_link.next = new Link<T>(item,null);
            return ret;
        }
    }
```

נעדכן את המתודה add כך שהיא תחזיר עותק חדש הכולל גם את האובייקט שניתן כפרמטר:

```
public Set<T> add(T obj) {
        return clone(obj);
}
```

המהדרין מגדירים גם את המחלקה כ-final ואף מעדכנים את הממשק בהתאם.

ג. לא. השכפול כבד מדי, וקשה למזג אחר כך עותקים.

ד. בתרחיש השני נדפיס עותק של המבנה כפי שהוא היה בתחילת המתודה, בתרחיש הראשון עשויים להתווסף בהמשך איברים נוספים לקבוצה וייתכן שגם הם יודפסו.
מאחר שאין מתודת remove לא ייתכן גם בתרחיש הראשון שנחשוב שיש איבר ולפני שנדפיס אותו הוא יימחק.
הייתי בוחר את הראשון כי הוא יותר יעיל ואין בו בעיה מהותית.

שאלה 3: תקשורת ובסיסי נתונים (35 נקודות)

א.

```
public interface MessageEncoderDecoder<T,R> {
    T decodeNextByte(byte nextByte);
    byte[] encode(R message);
}

public interface MessagingProtocol<T,R> {
    R process(T msg);
    boolean shouldTerminate();
}

public class ConnectionHandler<T,R> implements Runnable {

    private final MessagingProtocol<T,R> protocol;
    private final MessageEncoderDecoder<T,R> encdec;
    private final Socket sock;

    public ConnectionHandler(Socket sock, MessageEncoderDecoder<T,R> reader,
               MessagingProtocol<T,R> protocol) {
        this.sock = sock;
        this.encdec = reader;
        this.protocol = protocol;
    }

    @Override
    public void run() {

        try (   Socket sock = this.sock; //just for automatic closing
                BufferedInputStream in =
                   new BufferedInputStream(sock.getInputStream());
                BufferedOutputStream out =
                   new BufferedOutputStream(sock.getOutputStream())) {

            int read;
            while (!protocol.shouldTerminate() && (read = in.read()) >= 0) {
                T nextMessage = encdec.decodeNextByte((byte) read);
                if (nextMessage != null) {
```

```
                    R response = protocol.process(nextMessage);
                    if (response != null) {
                        out.write(encdec.encode(response));
                        out.flush();
                    }
                }
            }

        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

ב.

```
public interface Command<T> extends Serializable {
    Serializable execute(T data);
}

public class RemoteCommandInvocationProtocol<T>
     implements MessagingProtocol<Command, Serializable> {

    private T data;

    public RemoteCommandInvocationProtocol(T data) {
        this.data = data;
    }

    @Override
    public Serializable process(Command msg) {
        return msg.execute(data);
    }

    @Override
    public boolean shouldTerminate() {
        return false;
    }
}
```

ג.

```
final DB db = new DB();
class DBServer {
    public static void main(String[] args) {
        int port = IntegerParseInt(args[0]);
        new FixedThreadPoolServer(10,port,
            ()-> return new RemoteCommandInvocationProtocol<DB>(db),
            ()-> return new ObjectEncoderDecoder()).serve();
    }
}
```

.T

```
public class DBClient {

    public static void main(String[] args) throws Exception {
       RCIClient c = new RCIClient(args[0], 7777);
       c.send(new SelectCommand());
       System.out.println(c.receive());
    }
}

class SelectCommand implements Command<DB> {
     public Serializable execute(DB data) {
          return db.executeQuery("
             SELECT Orders.date, Orders.type, Orders.family, Volunteers.name
             FROM Orders INNER JOIN Volunteers
                         ON orders.volunteer = Volunteers.id
             WHERE Volunteers.hasLicense = TRUE
             ORDER BY Orders.date
         ");
     }
}
```